arXiv:submit/8337296 [cs.RO] 6 Nov 2026

LODOS-v2: An Efficient Primal Formulation of the Exact Frictional Contact Problem

Onur Beker
University of Tübingen

A. René Geist
University of Tübingen

Georg Martius
University of Tübingen

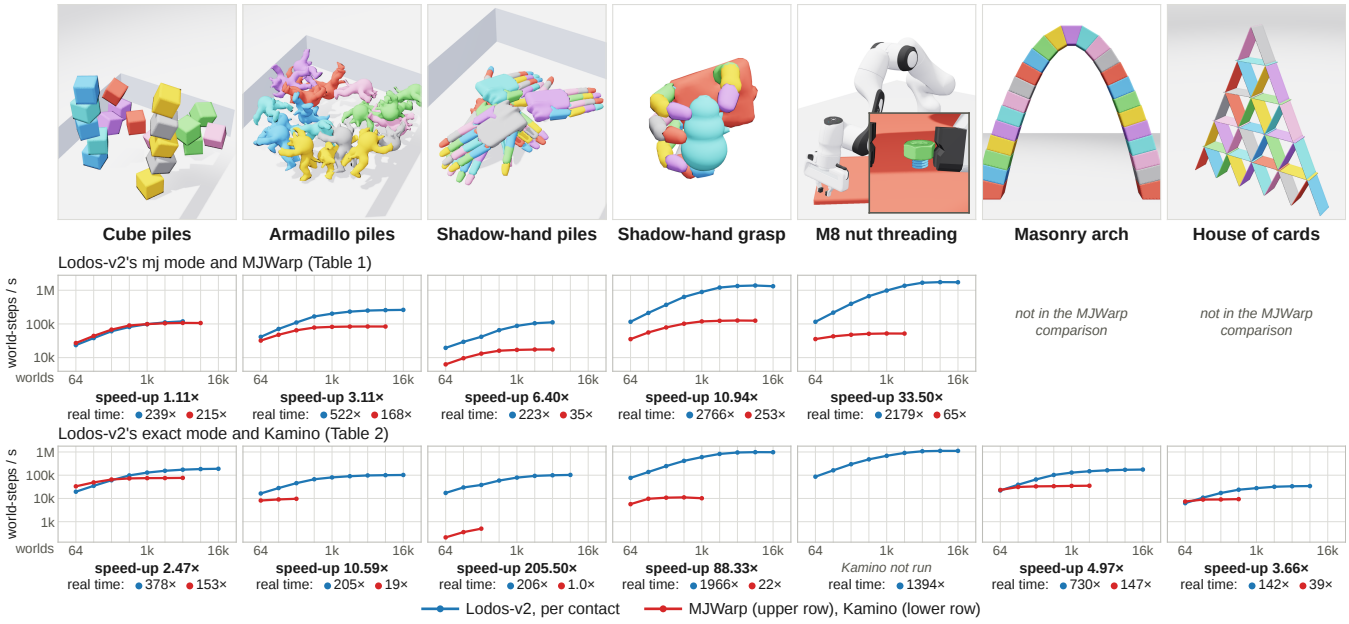


Figure 1: Benchmark scenes used to compare the throughput of Lodos-v2 against established simulation frameworks. Top: A representative frame from each scene, simulated using Lodos-v2's exact mode. Bottom: The speed-up achieved by Lodos-v2 (using the per contact parameterization) over each baseline in terms of maximum achievable throughput, computed from Tables 1 and 2.

Abstract

Lodos-v2 is a GPU-based simulation framework for massively parallel rigid-body dynamics with contact. It is specifically designed to achieve high throughput in scenes involving collisions among large numbers of detailed meshes. Its predecessor, Lodos-v1, had two main limitations. First, it solved the standard convex relaxation of cone-complementarity-based contact dynamics, thereby inheriting well-known physical artifacts, including lift-off and artificial compliance in the normal direction, as well as creep in the tangential direction. Second, its contact-group reparameterization scheme, which rescales spring-damper forces using a single factor per collision pair, could not be integrated with other constraints required by a full-fledged simulator, such as joint limits and welds. Lodos-v2 addresses both limitations through two main contributions. First, it introduces an efficient algorithm for solving the exact frictional contact problem by sequentially solving a cascade of convex relaxations in primal form within an outer proximal minimization loop. Second, it extends the contact-group reparameterization scheme of Lodos-v1 to the primal formulation of cone-complementarity-based contact dynamics, enabling its integration with the other constraints of a full-fledged simulator. Experiments demonstrate that these contributions result in throughput improvements over established rigid-body simulation frameworks. The source code is available at https://github.com/martius-lab/lodos.

1 Introduction

Massively vectorized simulation of rigid-body dynamics with contact [1–3] is essential for a range of robotics applications, including reinforcement learning (RL) [4, 5], sampling-based model-predictive control (MPC) [6, 7], and randomized smoothing [8, 9] for gradient-based optimization. Lodos-v1 [10] was developed to support such applications by enabling high-throughput simulation of scenes that contain large numbers of collision pairs between geometrically detailed meshes. Its two main contributions were (i) efficient broad-phase and narrowphase collision routines for vertex-SDF contacts and (ii) a "contact-group reparameterization" scheme that scales spring-damper contact forces using a single factor per collision pair, thereby making the number of solver decision variables and constraints independent of the number of contacts. Lodos-v2 preserves the collision detection framework of Lodos-v1 while substantially extending its treatment of contact dynamics. In particular:

- Lodos-v1 lacked a mechanism for integrating contact constraints with other constraint types commonly supported by general-purpose physics simulators, such as MuJoCo [11], including joint limits, weld constraints, and friction-loss constraints. Lodos-v2 overcomes this limitation, enabling it to reproduce MuJoCo's contact dynamics at higher simulation throughput.
- Lodos-v1 solved the convex relaxation of cone-complementarity-based contact dynamics introduced by Anitescu et al. [12], which

is also employed by MuJoCo. As a result, it inherits the known physical artifacts of this relaxation, including sliding contacts lifting-off, normal-direction compliance, and tangential creep that prevents contacts from remaining stationary under zero velocity and acceleration. Lodos-v2 addresses these limitations by introducing a novel primal-form solver for the exact cone complementarity problem, building on the work of Carpentier et al. [13]. This enables Lodos-v2 to reproduce the contact dynamics of Kamino [14] at higher simulation throughput.

This extended abstract provides a brief account of preliminary research results that improve on Lodos-v1 [10] that will eventually be incorporated into a full-length research paper.

2 Background

Notation: At each simulation step, the unconstrained rigid-body dynamics routine computes the generalized inertia matrix $M \in \mathbb{R}^{n_v \times n_v}$ and free acceleration $\ddot{q}_f \in \mathbb{R}^{n_v}$, while the collision detection routine identifies N contacts. Each contact i is characterized by a local frame $(\hat{n}_i, \hat{t}_{1,i}, \hat{t}_{2,i})$, a Jacobian $J_i \in \mathbb{R}^{3 \times n_v}$, a friction coefficient μ_i, and a signed distance $d_i \leq 0$. We further denote the Delassus matrix as $G = JM^{-1}J^\top$, the free contact velocity as $v_f = J\dot{q}_f$, where $\dot{q}_f = \dot{q} + \ddot{q}_f \Delta t$, the product of elliptic friction cones as $\mathcal{K} = \prod_i \mathcal{K}_{\mu_i}$, with $\mathcal{K}_\mu = \{f : \|f_t\| \leq \mu f_n\}$, the de Saxcé correction [15] as $\Gamma_i(\sigma) = \mu_i \|\sigma_{i,t}\| \hat{n}_i$, and the penetration resolution term as $\gamma_i = \gamma_{n,i}\hat{n}_i$.

The exact frictional contact problem: The exact frictional contact problem refers to finding contact forces that satisfy the following nonlinear cone complementarity condition (N-CCP) [13]:

$$\text{Find } f: \quad \mathcal{K} \ni f \perp (v^+ + \Gamma(v^+) + \gamma) \in \mathcal{K}^* \tag{1}$$

where $v^+ = v_f + Gf\Delta t$ is the end-of-step contact velocity and $\mathcal{K}^*$ is the dual friction cone. Given that v^+ is a linear function of f, an auxiliary velocity $\sigma = v^+$ can be introduced to decouple the nonlinear term $\Gamma(v^+)$ as $\Gamma(\sigma)$, resulting in an equivalent problem that only involves a linear cone complementarity condition:

$$\text{Find } f, \sigma: \quad \sigma = v_f + Gf\Delta t, \tag{2}$$

$$\mathcal{K} \ni f \perp (v^+ + \Gamma(\sigma) + \gamma) \in \mathcal{K}^* \tag{3}$$

When σ is fixed, (3) is equivalent to the optimality condition of the following quadratically constrained quadratic program (QCQP):

$$f^* = \arg\min_{f \in \mathcal{K}} \ \tfrac{1}{2} f^\top (G\Delta t) f + (v_f + \Gamma(\sigma) + \gamma)^\top f. \tag{4}$$

Therefore, the exact nonlinear cone complementarity problem can be equivalently transcribed as [13, 16]:

$$\text{Find } f^*, \sigma: \quad \sigma = v_f + Gf^*\Delta t, \tag{5}$$

$$f^* = \arg\min_{f \in \mathcal{K}} \ \tfrac{1}{2} f^\top (G\Delta t) f + (v_f + \Gamma(\sigma) + \gamma)^\top f. \tag{6}$$

Rank Deficiency and Proximal Minimization: Let us consider a generic convex optimization problem with a quadratic objective:

$$\min_{x \in C} \tfrac{1}{2} x^\top Q x + c^\top x \tag{7}$$

The case when Q is rank deficient is pathological, as there is no unique solution or Q^{-1}. The "proximal minimization" method [17, 18] provides a general framework for addressing such cases:

Algorithm 1 Proximal Minimization

```
Choose ρ > 0, D ≻ 0, x^0 ∈ C; set z ← x^0
while not converged do:
    for k = 1, ..., m do
        Do an optimization update on
        min_{x∈C} ½ x^⊤ Q x + c^⊤ x + ρ/2 ‖x − z‖²_D
        using any constrained optimization
        algorithm to obtain x*.
    end for
    z ← x*
end while
return x*
```

The algorithm repeatedly adds a quadratic penalty centered at the previous iterate z and applies m optimization updates to the resulting problem $\min_{x \in C} \frac{1}{2} x^\top Q x + c^\top x + \frac{\rho}{2} \|x - z\|_D^2$. The penalty adds $\rho D \succ 0$ to the Hessian, so every inner problem is strongly convex. At a fixed point $z = x^*$, the penalty vanishes, so x^* solves the original problem. The outer iteration converges to a minimizer even when Q is only positive semi-definite, provided the inner problems are solved accurately enough [18]. The main benefit of the framework is its flexibility: any constrained optimization algorithm can be wrapped in an outer proximal minimization loop to improve the conditioning of its inner updates.

Proximal Minimization for Frictional Contact: Adding the proximal penalty $\frac{\rho}{2}\|f - z\|_D^2$ to the QCQP objective (6) and collecting linear and quadratic terms together gives

$$\min_{f \in \mathcal{K}} \ \frac{1}{2} f^\top ((G + \frac{\rho}{\Delta t} D)\Delta t) f + (v_f + \Gamma(\sigma) + \gamma - \rho D z)^\top f. \tag{8}$$

Let us define $R = \frac{\rho}{\Delta t} D$ and $r = v_f + \Gamma(\sigma) + \gamma - \rho D z$ for notational brevity, and instead write:

$$\min_{f \in \mathcal{K}} \tfrac{1}{2} f^\top ((G + R)\Delta t) f + r^\top f \tag{9}$$

By cascading Eq. (5) and Eq. (6) into a fixed-point iteration scheme, and wrapping it in a proximal minimization loop, Carpentier et al. [13] propose the following solution routine for the exact frictional contact problem, as implemented in Kamino [14]:

Algorithm 2 Proximal ADMM with de Saxcé Correction

```
Choose ρ > 0, diagonal D ≻ 0, R = ρ/Δt D
Initialize z ∈ K, σ ← v_f
while not converged do:
    r ← v_f + Γ(σ) + γ − ρDz
    for k = 1, ..., m do
        Do an ADMM update on
        min_{f∈K} ½ f^⊤ ([(G + R)Δt]) f + r^⊤ f
        to obtain f*.
    end for
    z ← f*
    σ ← v_f + G f* Δt
end while
return f*
```

3 Method: An Efficient Primal-Form Solver

A transcription of the form Eq. (9) is called the dual-form of contact dynamics, with contact forces as the decision variables. Whenever $R \succ 0$, the solution of Eq. (9) is equivalent to the solution of the following nonlinear least-squares (NLS) transcription [19]:

$$\min_{\ddot{q}} \ \tfrac{1}{2}(\ddot{q}-\ddot{q}_f)^\top M(\ddot{q}-\ddot{q}_f) + \tfrac{1}{2}\sum_i R_{ii}^{-1}\|\Pi_{\mathcal{K}_i}(a_{\mathrm{ref},i} - J_i\ddot{q})\|^2, \quad (10)$$

with $a_{\mathrm{ref}} = J\ddot{q}_f - r/\Delta t$, which is also called the primal-form of the contact dynamics, with generalized acceleration $\ddot{q}$ as the main decision variable. Lodos solves the exact frictional contact problem by altering Algorithm 2 so that it operates on the NLS transcription and uses Newton updates rather than ADMM updates:

Algorithm 3 Proximal Newton with de Saxcé Correction

1: Choose $\rho > 0$, diagonal $D \succ 0$, $R = \frac{\rho}{\Delta t} D$
2: Initialize $z \in \mathcal{K}$, $\sigma \leftarrow v_f$
3: **while** not converged **do**:
4: $\quad r \leftarrow v_f + \Gamma(\sigma) + \gamma - \rho D z$
5: $\quad a_{\mathrm{ref}} \leftarrow J\ddot{q}_f - r/\Delta t$
6: $\quad$ **for** $k = 1, \ldots, m$ **do**
7: $\quad\quad$ Do a Newton update on
8: $\quad\quad$ $\min_{\ddot{q}} \ \frac{1}{2}(\ddot{q}-\ddot{q}_f)^\top M(\ddot{q}-\ddot{q}_f)$
9: $\quad\quad\quad + \frac{1}{2}\sum_{i=1}^{N} R_{ii}^{-1}\|\Pi_{\mathcal{K}_i}(a_{\mathrm{ref},\,i} - J_i\ddot{q})\|^2$
10: $\quad\quad$ to obtain $\ddot{q}^*$.
11: $\quad$ **end for**
12: $\quad f^* \leftarrow R^{-1}\Pi_{\mathcal{K}}(a_{\mathrm{ref}} - J\ddot{q}^*)$
13: $\quad z \leftarrow f^*$
14: $\quad \sigma \leftarrow v_f + G f^* \Delta t$
15: **end while**
16: **return** f^*

Lodos-v2 supports two simulation modes. The first, the "exact mode", implements Algorithm 3 to compute an exact solution to the frictional contact problem. The second, the "MuJoCo mode", instead solves Eq. (10), but uses the definitions of R and a_{ref} from MuJoCo's formulation [20], rather than those arising from the proximal minimization penalty and the de Saxcé correction.

4 Method: Contact-Group Reparameterization

Lodos-v1 [10] rescaled the spring-damper forces of a collision pair's contacts by one factor per pair, $f = F\alpha$ with $F \in \mathbb{R}^{3N\times|\mathcal{P}|}$ over the collision pairs $\mathcal{P}$ and $\alpha \succeq 0$. Lodos-v2 generalizes this to *contact groups*: a group $g \in \mathcal{G}$ is any subset of all contacts of one collision pair, and one variable $\lambda_g \in \mathbb{R}^{n_g}$ with $n_g \in \{1, 3\}$ sets the forces of all group members. A group takes one of three parameterizations:

Contact-Group Reparameterization

$$\text{per contact:}\quad f_i = C_i\lambda_i, \qquad \lambda_i \in \mathcal{K}_{\mu_i}, \quad (11.1)$$

$$\text{ray:}\quad f_i = a_i(\hat{n}_i - r_i\hat{t}_{1,i})\lambda_g, \quad \lambda_g \geq 0, \quad (11.2)$$

$$\text{cone:}\quad f_i = a_i C_i\lambda_g, \qquad \lambda_g \in \mathcal{K}_{\mu}. \quad (11.3)$$

For the per contact parameterization, no grouping is involved: each contact forms its own singleton group, such that $g = i$ and $n_g = 3$. In contrast, the ray ($n_g = 1$) and cone ($n_g = 3$) parameterizations support arbitrary groupings of contacts. In practice, these groupings are constructed at each time step by binning contacts into a fixed number of groups according to the wrench contributions associated with their normal and tangential velocities. For each contact i (independent of the grouping), the associated contact frame is $C_i = [\hat{n}_i\ \hat{t}_{1,i}\ \hat{t}_{2,i}] \in \mathbb{R}^{3\times3}$, with $\hat{t}_{1,i}$ along its tangential velocity $v_{i,t}$ and $\hat{t}_{2,i} = \hat{n}_i \times \hat{t}_{1,i}$. Its weight a_i and (in the case of the ray parameterization) its friction ratio r_i come from the spring-damper model used by [10], evaluated at the step's start and fixed for the step:

$$a_i = \frac{\bar{f}_{n,i}}{\sum_{j\in g}\bar{f}_{n,j}}, \qquad \bar{f}_{n,i} \propto f_d\Big(\frac{v_{i,n}}{\delta}\Big)\max(-d_i, 0), \quad (12)$$

with $f_d(x) = 1 - x$ for $x < 0$, $(x-2)^2/4$ for $0 \leq x < 2$ and 0 beyond, and $r_i = \mu\|v_{i,t}\|/(\epsilon^2 + \|v_{i,t}\|^2)^{1/2} < \mu$, a spring-damper model with dissipation and smooth stiction [21].

The QCQP Corresponding to Each Parameterization: Stacking the λ_g of all groups into a single vector $\lambda \in \mathbb{R}^{N_g}$, $N_g = \sum_g n_g$, each parameterization forms a linear map $f = B\lambda$ with $B \in \mathbb{R}^{3N\times N_g}$. Substituting this parameterization into (6) preserves the exact same QCQP form, this time in terms of λ rather than f:

$$\sigma = v_f + GB\lambda^*\Delta t,$$
$$\lambda^* = \arg\min_{\lambda\in\tilde{\mathcal{K}}} \ \tfrac{1}{2}\lambda^\top\big((\tilde{G}+\tilde{R})\Delta t\big)\lambda + \big(\tilde{v}_f + \tilde{\Gamma}(\sigma) + \tilde{\gamma}\big)^\top\lambda. \quad (13)$$

Here $\tilde{G} = B^\top GB = \tilde{J}M^{-1}\tilde{J}^\top \in \mathbb{R}^{N_g\times N_g}$, with the groups' Jacobian $\tilde{J} = B^\top J \in \mathbb{R}^{N_g\times n_v}$, whose block for group g is $\tilde{J}_g = \sum_{i\in g} B_i^\top J_i \in \mathbb{R}^{n_g\times n_v}$; $\tilde{R} = B^\top RB \in \mathbb{R}^{N_g\times N_g}$; $\tilde{v}_f = B^\top v_f$; $\tilde{\gamma} = B^\top\gamma$, which puts $\sum_{i\in g} a_i\gamma_{n,i}$ on λ_g's normal component; and $\tilde{\mathcal{K}} = \prod_g \tilde{\mathcal{K}}_g$, with $\tilde{\mathcal{K}}_g$ the group's set in (11). The problem overall has $3|\mathcal{G}| = 3N$ constraint rows for the per contact parameterization, $3|\mathcal{G}|$ for the cone parameterization, and $|\mathcal{G}|$ for the ray parameterization (plus all other constraint rows from non-contact constraints). The de Saxcé correction term, this time added to λ_g's normal component, is constructed from contact velocities σ_i as:

$$\tilde{\Gamma}_g(\sigma) = \begin{cases} \mu_i\|\sigma_{i,t}\| & \text{per contact } (g = \{i\}), \\ \sum_{i\in g} a_i r_i\,\hat{t}_{1,i}^\top\sigma_i & \text{ray}, \\ \mu\sum_{i\in g} a_i\|\sigma_{i,t}\| & \text{cone}. \end{cases} \quad (14)$$

5 Method: Implementation

The MJWarp codebase [2] provides an excellent, highly optimized unconstrained Newton solver for problems of the form given in Eq. (10). This solver, the nonsmooth Newton method of Todorov [19], is described in Algorithm 4 exactly as MJWarp implements it, with the same line search and convergence test. In MJWarp (and by extension in Lodos-v2), every simulation instance in a batched simulation runs Algorithm 4 independently in the same kernels. A converged world freezes and skips all later work, and the loop is a device-side while (a CUDA graph conditional node) over the unconverged worlds. Lodos-v2 builds upon MJWarp's solver as its computational core. Three new options change how lines 4–5 are computed, as discussed next.

Row Space: Lodos-v1 solved the dual form problem (9), and reduced its number of decision variables from $3N$ (i.e. three times the number of active contacts) to K (i.e. the number of active collision pairs) using contact-group reparameterization. This is still

Algorithm 4 MJWarp's Newton solver on the NLS transcription (10)

Input: rows (J, R, a_{ref}), cone $\mathcal{K}$, M, $\ddot{q}_f$; warm start $\ddot{q}_0$; ε = `tolerance`, $k_{\max}$ = `iterations`, $\ell_{\max}$ = `ls_iterations`, ε_ℓ = `ls_tolerance`

1: $\ddot{q} \leftarrow \ddot{q}_0$ ▷ last step's $\ddot{q}$ with `warm_start`, else $\ddot{q}_f$
2: $y \leftarrow a_{\text{ref}} - J\ddot{q}$; $f \leftarrow R^{-1}\Pi_{\mathcal{K}}(y)$; each row's state: inactive, quadratic, or the cone's middle zone
3: $g \leftarrow M(\ddot{q} - \ddot{q}_f) - J^\top f$
4: $H \leftarrow M + J^\top R^{-1}\, \partial\Pi_{\mathcal{K}}(y)\, J \in \mathbb{R}^{n_v \times n_v}$ ▷ `shared_hessian`: summed per world in shared memory
5: $d \leftarrow -H^{-1}g$ ▷ Cholesky; `row_space`: through the rows, H never formed
6: **for** $k = 1, \ldots, k_{\max}$ **do** ▷ while any world is unconverged
7: $\alpha \leftarrow$ exact line search on $\ell(\ddot{q} + \alpha d)$, the objective (10): bracket refinement until $|\partial\ell/\partial\alpha| < \max(\varepsilon\, \varepsilon_\ell\, \|d\|\, s,\ 10^{-6})$, at most $\ell_{\max}$ steps
8: $\ddot{q} \leftarrow \ddot{q} + \alpha d$; recompute y, f, the row states, g, H and d (lines 2–5)
9: **if** $(\ell_{\text{prev}} - \ell)/s < \varepsilon$ **or** $\|g\|/s < \varepsilon$ **or** $\frac{1}{2} g^\top H^{-1} g / s < \varepsilon$ **then**
10: freeze the world: converged
11: **end if**
12: **end for** ▷ a world still running at $k_{\max}$ freezes with the `ITERATIONS` flag
13: **return** $\ddot{q}$, f

more efficient than solving the primal form whenever $n_v \geq K$ (e.g. piles of free-bodies like a wall of bricks), but it does not readily lend itself into a combination with other types of constraints. The row-space option addresses this problem by formulating the problem in primal form, but using Woodbury identity to factor the corresponding Hessian in row-space (a matrix of size $K \times K$) rather than in generalized acceleration space (a matrix of size $n_v \times n_v$). Let $M = LL^\top$, $L \in \mathbb{R}^{n_v \times n_v}$ be the Cholesky factor of the mass matrix, lower triangular and block diagonal over the kinematic trees, and $W^\top W = R^{-1}\partial\Pi_{\mathcal{K}}(y)$, $W \in \mathbb{R}^{m \times 3N}$ be the Cholesky factor of the constraint-space Hessian (in line 4 of Algorithm 4), whose m rows now include the K constraints from Lodos-v1's contact-group reparameterization as well as other constraint types from MuJoCo. The Hessian H factors through L, and the Woodbury identity inverts its middle factor, which gives $H^{-1}g$ as:

$$
\begin{aligned}
M &= LL^\top, \quad L \in \mathbb{R}^{n_v \times n_v}, \\
G &= Z^\top Z, \ Z = L^{-1}J^\top \in \mathbb{R}^{n_v \times 3N}, \\
W^\top W &= R^{-1}\partial\Pi_{\mathcal{K}}(y), \quad W \in \mathbb{R}^{m \times 3N}, \\
H &= M + J^\top W^\top W J = L\big(\mathbf{I}_{n_v} + ZW^\top W Z^\top\big)L^\top, \\
\\
H^{-1}g &= L^{-\top}\big(\eta - ZW^\top \xi\big), \\
\eta &= L^{-1}g, \quad \xi = (\mathbf{I}_m + WGW^\top)^{-1} W Z^\top \eta,
\end{aligned}
\tag{15}
$$

Shared Hessian: Sums the sparse H per world in the shared memory of each streaming multiprocessor (SM) [22], if it fits.
Fused Newton: Runs Algorithm 4 and the proximal rounds around it as one kernel per world, therefore storing all the data (e.g. the solver state) in the shared memory of the SM, if it fits.
With these improvements, Algorithm 5 and Algorithm 6 describe the solution routines implemented by each mode of Lodos.

6 Results

This section summarizes benchmark experiments comparing Lodos-v2 with those solvers from the Newton [3] framework[1] whose contact dynamics its two modes reproduce: the MuJoCo mode against MJWarp (Newton's `SolverMuJoCo`), and the exact mode against Kamino [14] (Newton's `SolverKamino`). The main comparison metric is batched throughput, in world-steps per second, on one NVIDIA RTX 4080 with 16 GB of memory. Tables 1 and 2 give each configuration's best throughput and the speed-up of Lodos-v2 over each baseline, which Fig. 1 plots under the scenes. The exact experimental procedure, scenes, and settings can be found in the associated code release, which also provides utilities to reproduce the benchmark.

Scenes. Comparisons use the following scenes:

- `cube_walled_5/20`, `arm_walled_5/20`, `shadow_walled_5/20`: Scenes containing piles of 5/20 cubes, armadillos, or Shadow hands, respectively, dropped into a walled box. The cube scenes evaluate performance on free-floating rigid bodies with simple convex geometry, the armadillo scenes evaluate performance on free-floating rigid bodies with detailed nonconvex geometry, and the Shadow hand scenes evaluate performance on complex kinematic trees with detailed nonconvex geometry. Kamino is specifically designed for accurate simulation of kinematic structures with closed-loop topologies [14], a specialization that inherently limits its performance on kinematic trees. Accordingly, the cube and armadillo scenes provide a comparison between Kamino and Lodos-v2 under equivalent kinematic constraints, as neither involves kinematic trees or closed-loop structures. The Shadow hand scenes, by contrast, feature complex kinematic trees and are intended to highlight the complementary specializations of the two frameworks: Lodos-v2's focus on kinematic trees and Kamino's focus on closed-loop structures.
- `shadow_grasp`: a Shadow hand lifting a toy, replaying the position command from a prerecorded grasp. The scene replays an open-loop control sequence that is squeezing a delicate, four-fingered prismatic grasp. This means maintaining a stable grasp without slipping requires highly accurate friction modeling.
- `nist_threading_m8`: A Franka arm threads an M8 nut onto a bolt by replaying an open-loop control sequence recorded from an RL policy [23]. The fine screw threads of the nut and bolt require an accurate representation of detailed, non-convex collision geometry, making this a challenging simulation scenario.

[1] particularly the 1.7.0 version.

	Lodos-v2, mj mode				
scene	per contact	cone	ray	MJWarp	speed-up
cube_walled_5_polytope	1.61M @8192	–	–	1.11M @8192	1.45×
cube_walled_5_sdf	551k @4096	1.27M @8192	1.86M @8192	276k @2048	6.76×
arm_walled_5_polytope	985k @8192	–	–	163k @2048	6.04×
arm_walled_5_sdf	1.24M @4096	1.25M @8192	1.97M @8192	238k @2048	8.29×
shadow_grasp_polytope	1.18M @8192	–	–	432k @32768	2.73×
shadow_grasp_sdf	1.38M @8192	1.25M @8192	1.42M @8192	126k @4096	11.27×
shadow_walled_5_polytope	130k @8192	–	–	36.2k @2048	3.60×
shadow_walled_5_sdf	112k @4096	113k @8192	110k @8192	17.4k @4096	6.46×
nist_threading_m8	1.74M @8192	1.94M @8192	1.97M @8192	52k @1024	37.35×
cube_walled_20_polytope	394k @16384	–	–	256k @16384	1.54×
cube_walled_20_sdf	119k @4096	263k @16384	349k @8192	107k @4096	3.25×
arm_walled_20_polytope	178k @16384	–	–	46.3k @2048	3.85×
arm_walled_20_sdf	261k @16384	259k @16384	366k @16384	84k @4096	4.36×
shadow_walled_20_polytope	3.31k @512	–	–	2.59k @512	1.28×
shadow_walled_20_sdf	2.88k @512	2.3k @512	2.36k @512	626 @64	4.60×

Table 1: Lodos-v2's MuJoCo mode against MJWarp. Each configuration's best throughput, in world-steps per second at the batch size after @ (k = 10^3, M = 10^6), and the speed-up of Lodos-v2's fastest configuration over MJWarp's best.

	Lodos-v2, exact mode				
scene	per contact	cone	ray	Kamino	speed-up
cube_walled_5_polytope	711k @8192	–	–	589k @4096	1.21×
cube_walled_5_sdf	225k @2048	596k @8192	1.08M @4096	–	1.83×‡
arm_walled_5_polytope	408k @8192	–	–	58.3k @128	6.99×
arm_walled_5_sdf	523k @4096	544k @4096	1.12M @8192	–	19.26×‡
shadow_grasp_polytope	983k @8192	–	–	11.1k @512	88.33×
shadow_grasp_sdf	660k @8192	675k @8192	1.02M @8192	–	60.64×‡
shadow_walled_5_polytope	103k @8192	–	–	501 @256	205.50×
shadow_walled_5_sdf	65.3k @4096	69.4k @8192	74.4k @4096	–	148.50×‡
nist_threading_m8	1.12M @8192	1.25M @8192	1.45M @8192	–	–
masonry_arch_polytope	175k @16384	–	–	35.3k @2048	4.97×
masonry_arch_sdf	44.3k @2048	34.8k @1024	21.4k @1024	–	1.26×‡
house_of_cards_polytope	34.2k @8192	–	–	9.34k @512	3.66×
house_of_cards_sdf	7.89k @2048	18.6k @4096	16.8k @2048	–	1.99×‡
cube_walled_20_polytope	189k @16384	–	–	76.6k @4096	2.47×
cube_walled_20_sdf	61k @4096	148k @16384	199k @8192	–	2.60×‡
arm_walled_20_polytope	102k @16384	–	–	9.66k @256	10.59×
arm_walled_20_sdf	143k @16384	142k @16384	216k @16384	–	22.31×‡
shadow_walled_20_polytope	2.62k @1024	–	–	left out	–
shadow_walled_20_sdf	1.74k @512	1.58k @512	1.56k @512	–	–

Table 2: Lodos-v2's exact mode against Kamino. The best throughput achieved by each configuration and the speed-up of Lodos-v2's fastest configuration. Kamino is evaluated only on the polytope versions of the scenes. The symbol ‡ indicates comparisons between Lodos-v2 on the original surfaces and Kamino on the corresponding polytope versions. The threading task has no polytope version, and therefore does not include a Kamino baseline. Similarly, Kamino was excluded from the twenty-hands scene because benchmarking was prohibitively slow (at a batch size of 16, each simulation step took 7.2 s).

- `masonry_arch`: A masonry arch composed of 25 stones [24]. Maintaining the arch's stability requires physically accurate modeling of frictional interactions between the stones.
- `house_of_cards`: A house of 40 cards [24]. It likewise requires accurately modeling frictional interactions between the cards.

Each scene but `nist_threading_m8` comes in two versions, sharing the same bodies, inertias and starts. For the `_sdf` version, Lodos-v2 collides the bodies using the collision pipeline of Lodos-v1 (i.e. vertices against signed distance fields), and Newton using its mesh-SDFs collision pipeline (with contact reduction on). For the `_polytope` version, both collide them as polytopes obtained from convex decomposition [25]. The threading scene only has the `_sdf` version as a convex decomposition does not provide adequate geometric fidelity given the fine screw threads. The Kamino baseline is evaluated only on the `_polytope` variants, as it was prohibitively slow to benchmark with the large number of contact points generated by Newton's mesh-SDF collision pipeline.

Settings. Lodos-v2 runs each `_sdf` scene under all three contact-group parameterizations (i.e. per contact, cone and ray), and each `_polytope` scene only using the per contact parameterization. In the MJWarp comparison, both simulators use the same contact model parameters (solref 5 ms with damping ratio 1; 2.5 ms on the threading), the same parameters for the Newton routine (100 iterations, tolerance 10^{-6}, 50 line-search iterations), elliptic cones and the implicitfast integrator. In the Kamino comparison, Lodos-v2's exact mode performs $n = 2$ proximal rounds with $\hat{\rho} = 0.05$ and $\alpha_B = 0.05$ (Algorithm 5), while Kamino uses its default Newton solver settings with the number of iterations capped at 20.[2] These settings are chosen such that both solvers pass the physical faithfulness benchmark of [24]. This benchmark comprises scenes specifically designed to expose the limitations of simulators based on convex relaxations (such as MuJoCo), which produce physical artifacts, while requiring simulators that solve the exact frictional contact problem to achieve physically faithful results.

Measurement. Every world in a batch starts from its own unique state to eliminate confounders due to caching: a pile from a random stack of its own, the other scenes from their start with each free body randomly perturbed by up to 0.05 mm. Each configuration's memory capacities (e.g. for the pre-allocated arrays that store contact and collision data) are 1.5 times a world's largest demand (over 16 randomized starts) times the batch size. A run times the whole simulated span after its step has been captured: 3 s of a pile (the minimum time needed for the pile to settle), 5 s of the grasp and the threading (the duration of the pre-recorded control trajectories), 2 s of the arch and the house of cards (the minimum time needed for the scene to settle). The batch size doubles from 64 worlds until the throughput drops or the batch no longer fits.

Reproducing and viewing the results. The code for running both benchmarks can be found in the folder `bench/` of the associated codebase (under `bench/lodos_mjwarp` and `bench/lodos_kamino`). The file `bench/README.md` describes the steps needed to reproduce every number in Tables 1 and 2, and each benchmark's report gives the full settings, the capacities, the execution options and the throughput at every batch size. The file `viewer/README.md` shows how to open each scene in the live viewer that comes with the codebase, under all three simulators.

7 Conclusion

In this extended abstract, we have presented Lodos-v2, a framework for high-throughput simulation of rigid-body dynamics with contact in scenes involving large numbers of collision pairs and detailed geometries. An immediate direction for future work is to demonstrate the framework's potential to accelerate applications such as reinforcement learning (RL), model predictive control (MPC), and randomized smoothing.

[2] For the hand scenes, the cap is increased to 200 iterations, as fewer iterations resulted in physical artifacts.

References

[1] Viktor Makoviychuk, Lukasz Wawrzyniak, Yunrong Guo, et al. Isaac gym: High performance gpu-based physics simulation for robot learning, 2021. arXiv:2108.10470. 1

[2] Google DeepMind and NVIDIA. MuJoCo Warp (MJWarp), 2025. 3

[3] The Newton Contributors. Newton: GPU-accelerated physics simulation for robotics and simulation research, April 2025. 1, 4

[4] Jemin Hwangbo, Joonho Lee, Alexey Dosovitskiy, Dario Bellicoso, Vassilios Tsounis, Vladlen Koltun, and Marco Hutter. Learning agile and dynamic motor skills for legged robots. *Science robotics*, 4(26):eaau5872, 2019. 1

[5] Arthur Allshire, Mayank Mittal, Varun Lodaya, Viktor Makoviychuk, Denys Makoviichuk, Felix Widmaier, Manuel Wüthrich, Stefan Bauer, Ankur Handa, and Animesh Garg. Transferring dexterous manipulation from gpu simulation to a remote real-world trifinger. In *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 11802–11809. IEEE, 2022. 1

[6] Vince Kurtz. Hydrax: Sampling-based model predictive control on gpu with jax and mujoco mjx, 2024. https://github.com/vincekurtz/hydrax. 1

[7] Albert H. Li, Preston Culbertson, Vince Kurtz, and Aaron D. Ames. Drop: Dexterous reorientation via online planning, 2024. arXiv:2409.14562. 1

[8] Hyung Ju Terry Suh, Tao Pang, and Russ Tedrake. Bundled gradients through contact via randomized smoothing. *IEEE Robotics and Automation Letters*, 7(2): 4000–4007, 2022. 1

[9] Tao Pang, HJ Terry Suh, Lujie Yang, and Russ Tedrake. Global planning for contact-rich manipulation via local smoothing of quasi-dynamic contact models. *IEEE Transactions on robotics*, 39(6):4691–4711, 2023. 1

[10] Onur Beker, Andreas René Geist, and Georg Martius. LODOS: A lightweight simulator designed for contact-rich interactions with detailed meshes, 2026. Under review. 1, 2, 3

[11] Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033. IEEE, 2012. doi: 10.1109/IROS.2012.6386109. 1

[12] Mihai Anitescu. Optimization-based simulation of nonsmooth rigid multibody dynamics. *Mathematical Programming*, 105:113–143, 2006. 1

[13] Justin Carpentier, Louis Montaut, and Quentin Le Lidec. From compliant to rigid contact simulation: a unified and efficient approach, 2024. arXiv:2405.17020. 2

[14] Vassilios Tsounis, Guirec Maloisel, Christian Schumacher, Ruben Grandia, Agon Serifi, David Müller, Chris Amevor, Tobias Widmer, and Moritz Bächer. Kamino: Gpu-based massively parallel simulation of multi-body systems with challenging topologies. *arXiv preprint arXiv:2603.16536*, 2026. 2, 4

[15] Géry de Saxcé and Zhi-Qiang Feng. New inequality and functional for contact with friction: The implicit standard material approach. *Mechanics of Structures and Machines*, 19(3):301–325, 1991. doi: 10.1080/08905459108905146. 2

[16] Vincent Acary, Maurice Brémond, and Olivier Huber. On solving contact problems with Coulomb friction: formulations and numerical comparisons. In Remco I. Leine, Vincent Acary, and Olivier Brüls, editors, *Advanced Topics in Nonsmooth Dynamics*, pages 375–457. Springer, Cham, 2018. doi: 10.1007/978-3-319-75972-2_10. 2

[17] Jean Jacques Moreau. Proximité et dualité dans un espace hilbertien. *Bulletin de la Société Mathématique de France*, 93:273–299, 1965. doi: 10.24033/bsmf.1625. 2

[18] R. Tyrrell Rockafellar. Monotone operators and the proximal point algorithm. *SIAM Journal on Control and Optimization*, 14(5):877–898, 1976. doi: 10.1137/0314056. 2

[19] Roboti LLC. MuJoCo Pro 1.5: Release notes. Official software release notes, 2017. Release dated 23 April 2017. Documents the acceleration-space Newton solver and the reduced primal formulation; accessed 5 October 2026. 3

[20] Emanuel Todorov. Convex and analytically-invertible dynamics with contacts and constraints: Theory and implementation in mujoco. In *2014 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6054–6061. IEEE, 2014. 3

[21] Vince Kurtz, Alejandro Castro, Aykut Özgün Önol, and Hai Lin. Inverse dynamics trajectory optimization for contact-implicit model predictive control. *The International Journal of Robotics Research*, 45(1):23–40, 2026. 3

[22] NVIDIA Corporation. Cuda programming guide, release 13.2. https://docs.nvidia.com/cuda/cuda-programming-guide/pdf/cuda-programming-guide.pdf, 2026. 4

[23] O. Zhang, M. G. Castro, P. Yin, I. Dagnino, A. Gupta, R. Scalise, and B. Boots. A balanced data diet: Addressing the exploration bottleneck in mega-scale rl for robot control. *arXiv preprint arXiv:2610.12465*, 2026. 4

[24] Hongcheng Song, Ye Fan, Uri M. Ascher, and Dinesh K. Pai. A splitting architecture for exact reduced Coulomb friction. *Computer Graphics Forum*, 45(8), 2026. doi: 10.1111/cgf.70576. Proc. ACM SIGGRAPH / Eurographics Symposium on Computer Animation (SCA). 5, 6

[25] Xinyue Wei, Minghua Liu, Zhan Ling, and Hao Su. Approximate convex decomposition for 3d meshes with collision-aware concavity and tree search. *ACM Transactions on Graphics (TOG)*, 41(4):1–18, 2022. 5

Algorithm 5 The exact mode of Lodos: proximal minimization with de Saxcé correction around Algorithm 4

Input: $\hat{\rho}$ = `prox_rho`, n = `prox_iters_per_qp` ≥ 1, ε_z = `prox_tol_z`, ε_σ = `prox_tol_sigma`, α_B = `baumgarte_alpha`, s_B = `baumgarte_slop`

$\mathcal{K} \leftarrow$ the elliptic cones; impratio $\leftarrow 1$
for each contact i **do** ▷ `_prox_init`, all contacts in parallel
  $R_{ii} \leftarrow \hat{\rho}\, w_i$
  $\gamma_{n,i} \leftarrow \alpha_B \min\big(0, (d_i + s_B)/\Delta t\big) + \max(0, d_i/\Delta t)$ ▷ 0 without `baumgarte`
  $z_i \leftarrow \Pi_{\mathcal{K}_i}$(its cached anchor) if `warm_start` matches it in the cross-step cache, else 0
  $\sigma_i \leftarrow J_i(\dot{q} + \Delta t\, \ddot{q}_0)$ ▷ at the warm start
end for
$(n_o, n_i) \leftarrow (n, 1)$ with `refresh_de_saxce_per_prox`, else $(1, n)$
for $o = 1, \ldots, n_o$ **do** ▷ each round after the first is skipped once every world is frozen
  $\Gamma_i \leftarrow \mu_i \|\sigma_{i,t}\|\, \hat{n}_i$ ▷ 0 without `de_saxce`
  for $j = 1, \ldots, n_i$ **do**
    $a_{\mathrm{ref},i,n} \leftarrow -\big(v_{i,n} + \gamma_{n,i} + \Gamma_{i,n}\big)/\Delta t + R_{ii} z_{i,n}$; $a_{\mathrm{ref},i,t} \leftarrow -v_{i,t}/\Delta t + R_{ii} z_{i,t}$ ▷ $v_i = J_i \dot{q}$
    $\ddot{q} \leftarrow$ Algorithm 4 on these rows, from the current $\ddot{q}$
    $f^* \leftarrow R^{-1} \Pi_{\mathcal{K}}(a_{\mathrm{ref}} - J\ddot{q})$; $\Delta z \leftarrow \max|f^* - z|$; $z \leftarrow f^*$
    if $n_i > 1$ **and** $\Delta z \leq \varepsilon_z(1 + \max|f^*|)$ **then** freeze the world
    end if
  end for
  if `refresh_de_saxce_per_prox` **then**
    $\sigma \leftarrow J(\dot{q} + \Delta t\, \ddot{q})$; $\Delta\Gamma \leftarrow \max_i|\Gamma_i(\sigma) - \Gamma_i|$
    if $\Delta z \leq \varepsilon_z(1 + \max|f^*|)$ **and** $\Delta\Gamma \leq \varepsilon_\sigma(1 + \max|\Gamma|)$ **then** freeze the world
    end if
  end if
end for
with `warm_start`: write each contact's anchor into the cross-step cache (`_prox_cache_fill`)
return $\ddot{q}$, f^*

Algorithm 6 The MuJoCo mode of Lodos

Input: the model's cone and impratio; each row's `solref`, `solimp`; $\hat{\rho}$, n = `prox_iters_per_qp`, `de_saxce`

for each contact i **do**
  $R_{ii} \leftarrow w_i\,(1 - \mathrm{imp}_i)/\mathrm{imp}_i$, its friction rows' divided by impratio
  $a_{\mathrm{ref},i} \leftarrow k_i\, \mathrm{imp}_i\, |d_i|\, \hat{n}_i - b_i\, v_i$ ▷ MuJoCo's spring-damper reference
end for
if $n = 0$ **and not** `de_saxce` **then** ▷ MuJoCo's problem; nothing overridden
  $\ddot{q} \leftarrow$ Algorithm 4 on these rows, on the model's cone ▷ elliptic: $\mu_r = \mu/\sqrt{\mathrm{impratio}}$
else ▷ the proximal path on MuJoCo's soft rows (`opt.prox_soft`), elliptic cone
  $\rho_i \leftarrow \hat{\rho}\, w_i$ if $n > 0$, else 0 ▷ without a loop: one round, carrying the term alone
  in Algorithm 5, lines 9–23 with $R'_{ii} \leftarrow R_{ii} + \rho_i$ and the references
  $a'_{\mathrm{ref},i,n} \leftarrow a_{\mathrm{ref},i,n} - b_i\, \Gamma_{i,n} + \rho_i z_{i,n}$, $a'_{\mathrm{ref},i,t} \leftarrow a_{\mathrm{ref},i,t} + \rho_i z_{i,t}$, and no Baumgarte bias
end if
return $\ddot{q}$, f